PYTHON FOR BEGINNERS · SESSION 2

Variables and Data Types

Store values in variables, learn the data types, take input and convert between types.

Recap

Ask the class:

- What did we run last class?
- What does `print()` do?
- What is the difference between a comment and a statement?

What is a variable?

A variable is a **named container that stores a value in memory**.

Think of a labelled box: the label is the **variable name** and the thing inside is the **value**.

variables.py

```
name = "Aman"
age = 25
print(name)
print(age)
```

OUTPUT

```
Aman
25
```

The = sign

`=` is the **assignment operator**. It means “store the right side in the left side”. It does **not** mean “equal” as in maths.

Changing a variable

A variable can be changed at any time. The newest value replaces the old one.

```
age = 25
age = 26
print(age)
```

OUTPUT

```
26
```

Ask

What will `print(age)` show after `age = 26`?

Assign several at once

```
a, b, c = 1, 2, 3
x = y = 0
print(a, b, c)
print(x, y)
```

OUTPUT

```
1 2 3
0 0
```

Rules for naming variables

- Must start with a **letter or underscore** (`_`), never a digit.
- Can contain **letters, digits and underscores** only, with no spaces or special characters.
- **Case-sensitive:** `Name` and `name` are different variables.
- Cannot be a **keyword** such as `if`, `for`, `class` or `True`.

Valid and invalid names

VALID	INVALID
`student_name`	`1student`
`_count`	`my name`
`marks2`	`total-marks`
`Age`	`class`

Naming convention

Use **snake_case** (`total_marks`) and give variables **meaningful names** (`price`, not `p`).

Ask

Is `my_age` valid? What about `my-age`?

Data types

Python decides the type **from the value**. Check it with `type()`.

TYPE	MEANING	EXAMPLE
`int`	whole numbers	`10`, `-5`, `0`
`float`	decimal numbers	`3.14`, `-0.5`
`str`	text	`"Python"`, `'A'`
`bool`	True or False	`True`, `False`
`NoneType`	no value	`None`

Checking the type

```
a = 10
b = 3.14
c = "Hello"
d = True
e = None

print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

Things to remember

- `10` is an `int`, but `"10"` is a `str`.
- `True` and `False` start with a **capital letter**.
- Python is **dynamically typed**, so one variable can hold different types at different times.

```
x = 5
print(type(x))
x = "five"
print(type(x))
```

OUTPUT

```
<class 'int'>
<class 'str'>
```

Ask

What is the type of `"3.14"`? Of `3.0`?

Taking input from the user

Run this one in VS Code or a terminal

```
name = input("Enter your name: ")
print("Hello", name)
```

Important

`input()` **always returns a string**, even when the user types a number.

input() always gives a string

Run this one in VS Code or a terminal

```
a = input("Enter a number: ")
print(type(a))
```

Enter `5` and the type is still `str`.

Type conversion (type casting)

Convert one type into another with `int()`, `float()`, `str()` and `bool()`.

Run this one in VS Code or a terminal

```
age = int(input("Enter your age: "))
price = float(input("Enter price: "))
print(age + 1)
print(price * 2)
```

More examples

```
print(int("25"))
print(float(7))
print(str(100))
print(int(9.99))
print(bool(0))
print(bool("hi"))
```

OUTPUT

```
25
7.0
100
9
False
True
```

- `int(9.99)` gives `9`. It **truncates**, it does not round.
- `bool(0)`, `bool("")` and `bool(None)` are `False`. **Everything else is True.**

A common error

Text and numbers cannot be added. Run this to see the `TypeError`, then fix it with `int(a) + 5`.

Try it: run, read the error, then fix it

```
a = "5"          # input() gives you text like this
print(a + 5)
```

ERROR

```
  File "your code", line 2, in <module>
TypeError: can only concatenate str (not "int") to str
```

Ask

Why does `"5" + "5"` give `55`?

Basic operators

Arithmetic

```
a, b = 10, 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

OUTPUT

```
13
7
30
3.3333333333333335
3
1
1000
```

- `/` **always returns a float**.
- `//` is **floor division**.
- `%` is the **remainder**.
- `**` is the **power**.

Assignment operators

```
x = 10
x += 5
x -= 3
x *= 2
x /= 4
print(x)
```

OUTPUT

```
6.0
```

Operator precedence

Python works in this order: `()`, then `**`, then `* / // %`, then `+ -`.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

OUTPUT

```
14
20
```

Useful extras

```
name = "Aman"
print(5 + 3, "is the sum")
print("Age:", 25)
print("Name: " + name)
print("Age: " + str(25))
```

OUTPUT

```
8 is the sum
Age: 25
Name: Aman
Age: 25
```

Remember

Mixing a string and a number with `+` needs `str()`. Using a **comma** in `print()` does not.

Practice questions

1. Store your name, age, height and student status in variables and print each with its type.
2. Take two numbers from the user and print their sum, difference, product and division.
3. Take the radius of a circle and print its area (`3.14 * r * r`).
4. Take marks in 3 subjects and print the total and average.
5. Swap two variables without using a third variable (`a, b = b, a`).
6. Take a temperature in Celsius and convert it to Fahrenheit (`F = C * 9/5 + 32`).
7. Find the error and fix it (see the next slide).

Find the error and fix it

Your turn: fix it and press Run

```
age = "20"     # imagine this came from input()
print(age + 1)
```

Try the swap

Your turn: swap a and b, then press Run

```
a = 5
b = 10
# swap a and b without a third variable
print(a, b)
```

Homework and wrap-up

- Write a simple **bill calculator**: take the item price and quantity, and print the total.
- Write a program that takes **name, age and city**, and prints one formatted introduction line.
- Revise the **naming rules** and **data types**.

Next class

Strings and operators.